mini_project_B

October 28, 2026

1 MINI_PROJECT

2 Time-Series / Contextual Analysis, wherever applicable

3 Part A — Time Indexing and Exploratory Analysis

[2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print("liberary loaded successfully")
```

```
liberary loaded successfully
```

[3]:
```
df = pd.read_csv("mini_project.csv")
print("file readed successfully")
```

```
file readed successfully
```

[4]:
```
print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())
```

```
Dataset shape: (100000, 13)

Column names:
['timestamp', 'machine_id', 'temperature', 'vibration', 'humidity', 'pressure',
'energy_consumption', 'machine_status', 'anomaly_flag',
'predicted_remaining_life', 'failure_type', 'downtime_risk',
'maintenance_required']
```

[6]:
```
# 2. Convert timestamp to datetime
df["timestamp"] = pd.to_datetime(df["timestamp"], errors="coerce")

# Remove rows with invalid timestamps
df = df.dropna(subset=["timestamp"])
```

[7]:
```
# 3. Select one machine and one numerical parameter
MACHINE_ID = df["machine_id"].mode()[0]
PARAMETER = "temperature"

# Filter data for the selected machine
data = df[df["machine_id"] == MACHINE_ID].copy()

# Convert selected parameter to numeric
data[PARAMETER] = pd.to_numeric(data[PARAMETER], errors="coerce")

# Sort data chronologically
data = data.sort_values("timestamp")
```

[8]:
```
# 4. Check duplicate timestamps
print("\nSelected machine:", MACHINE_ID)
print("Selected parameter:", PARAMETER)
print("Duplicate timestamps:", data["timestamp"].duplicated().sum())

# Remove duplicate timestamps for this machine
data = data.drop_duplicates(subset=["timestamp"], keep="first")
```

```
Selected machine: 23
Selected parameter: temperature
Duplicate timestamps: 0
```

[9]:
```
# Check missing values
print("\nMissing values:")
print(data[["timestamp", PARAMETER]].isna().sum())

# Remove missing parameter values for analysis
data = data.dropna(subset=[PARAMETER])

# Set timestamp as index
data = data.set_index("timestamp")
```

```
Missing values:
timestamp      0
temperature    0
dtype: int64
```

[10]:
```
# 5. Find sampling interval and observation period
time_gaps = data.index.to_series().diff().dropna()

print("\nObservation period:")
print("Start:", data.index.min())
```

```
print("End:", data.index.max())

print("\nSampling interval:")
print(time_gaps.value_counts().head())

if len(time_gaps) > 0:
    print("Median interval:", time_gaps.median())

print("\nNumber of observations:", len(data))
```

```
Observation period:
Start: 2025-01-01 00:08:00
End: 2025-03-11 10:36:00

Sampling interval:
0 days 00:04:00    51
0 days 00:11:00    43
0 days 00:02:00    40
0 days 00:03:00    40
0 days 00:01:00    39
Name: timestamp, dtype: int64
Median interval: 0 days 00:33:00

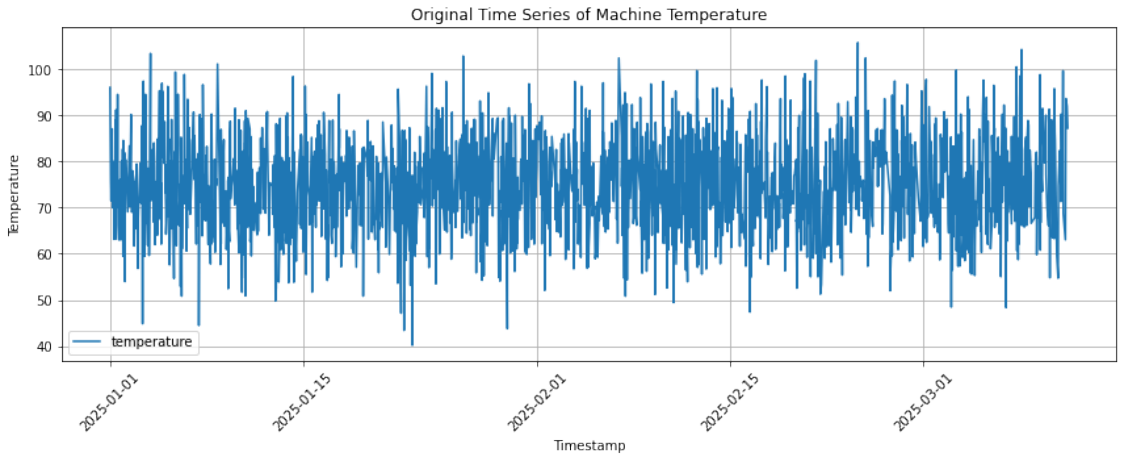
Number of observations: 2092
```

[11]:
```
# 6. Descriptive statistics
print("\nDescriptive statistics:")
print(data[PARAMETER].describe())

# 7. Plot original time series
plt.figure(figsize=(12, 5))
plt.plot(data.index, data[PARAMETER], label=PARAMETER)
plt.xlabel("Timestamp")
plt.ylabel("Temperature")
plt.title("Original Time Series of Machine Temperature")
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

```
Descriptive statistics:
count    2092.000000
mean       74.785942
std         9.959972
min        40.180000
```

```
25%         67.955000
50%         74.710000
75%         81.582500
max        105.770000
Name: temperature, dtype: float64
```



4 Part B — Rolling Statistics, Moving Averages and Differencing

[13]:
```python
# Load and prepare the data
df["timestamp"] = pd.to_datetime(df["timestamp"], errors="coerce")
df = df.dropna(subset=["timestamp"])

MACHINE_ID = df["machine_id"].mode()[0]
PARAMETER = "temperature"

data = df[df["machine_id"] == MACHINE_ID].copy()
data[PARAMETER] = pd.to_numeric(data[PARAMETER], errors="coerce")
data = data.sort_values("timestamp")
data = data.drop_duplicates(subset=["timestamp"], keep="first")
data = data.dropna(subset=[PARAMETER])
data = data.set_index("timestamp")

series = data[PARAMETER]
```

[14]:
```python
# 1. Rolling mean and standard deviation
data["rolling_mean_5"] = series.rolling(window=5).mean()
data["rolling_std_5"] = series.rolling(window=5).std()

data["rolling_mean_10"] = series.rolling(window=10).mean()
data["rolling_std_10"] = series.rolling(window=10).std()
```

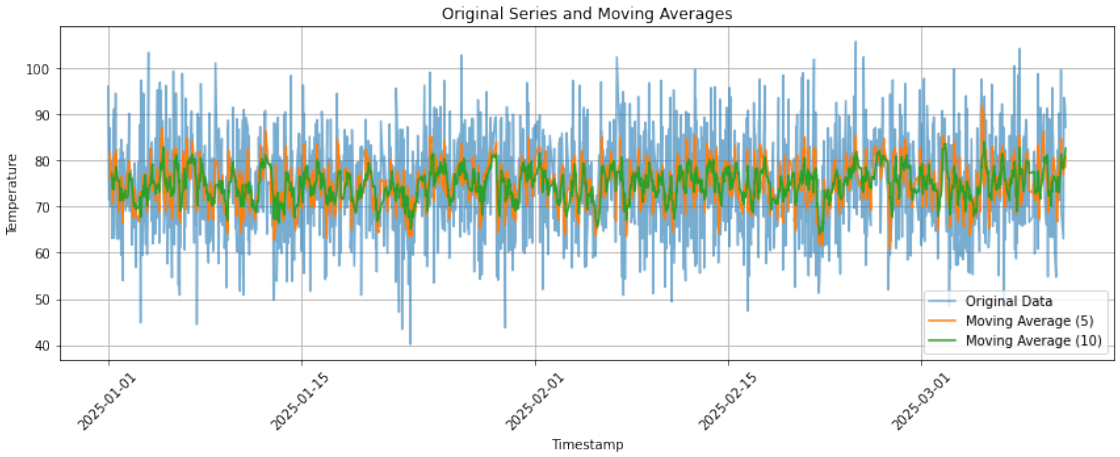
```
print("Rolling Statistics:")
print(data[[PARAMETER, "rolling_mean_5",
            "rolling_std_5", "rolling_mean_10",
            "rolling_std_10"]].head(15))
```

```
Rolling Statistics:
                     temperature  rolling_mean_5  rolling_std_5  \
timestamp
2025-01-01 00:08:00        96.04             NaN            NaN
2025-01-01 01:22:00        75.98             NaN            NaN
2025-01-01 02:04:00        71.47             NaN            NaN
2025-01-01 02:38:00        77.00             NaN            NaN
2025-01-01 03:39:00        87.08          81.514       9.923058
2025-01-01 04:29:00        70.00          76.306       6.704840
2025-01-01 06:14:00        80.01          77.112       6.895373
2025-01-01 06:29:00        71.23          77.064       6.945123
2025-01-01 07:00:00        79.15          77.494       7.006456
2025-01-01 07:06:00        63.15          72.708       6.994492
2025-01-01 07:43:00        72.93          73.294       6.831741
2025-01-01 08:27:00        67.14          70.720       6.048355
2025-01-01 09:13:00        81.79          72.832       7.847173
2025-01-01 09:47:00        88.92          74.786      10.560877
2025-01-01 09:53:00        63.19          74.794      10.549869

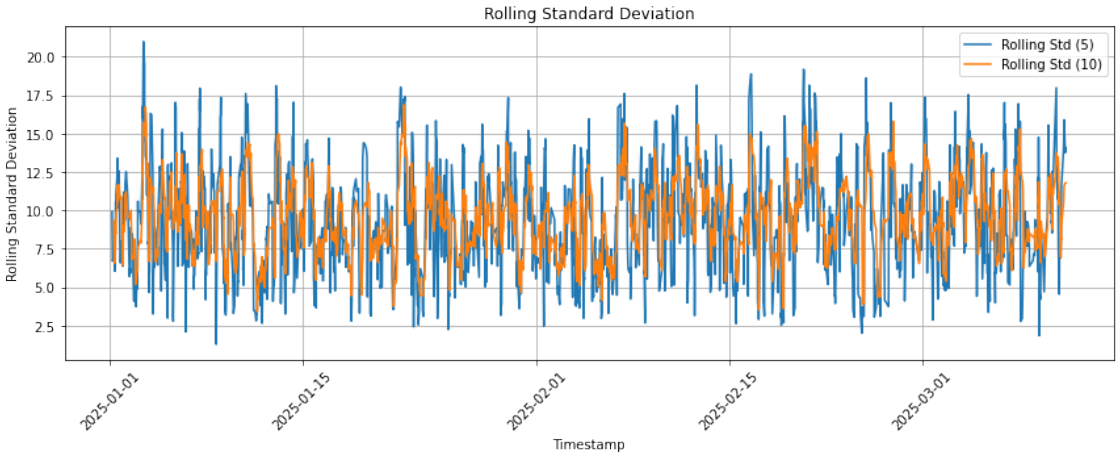
                     rolling_mean_10  rolling_std_10
timestamp
2025-01-01 00:08:00              NaN             NaN
2025-01-01 01:22:00              NaN             NaN
2025-01-01 02:04:00              NaN             NaN
2025-01-01 02:38:00              NaN             NaN
2025-01-01 03:39:00              NaN             NaN
2025-01-01 04:29:00              NaN             NaN
2025-01-01 06:14:00              NaN             NaN
2025-01-01 06:29:00              NaN             NaN
2025-01-01 07:00:00              NaN             NaN
2025-01-01 07:06:00           77.111        9.329905
2025-01-01 07:43:00           74.800        6.575971
2025-01-01 08:27:00           73.916        6.981398
2025-01-01 09:13:00           74.948        7.333530
2025-01-01 09:47:00           76.140        8.568829
2025-01-01 09:53:00           73.751        8.509923
```

[15]:
```
# 2. Plot original series and moving averages
plt.figure(figsize=(12, 5))
plt.plot(data.index, series, label="Original Data", alpha=0.6)
plt.plot(data.index, data["rolling_mean_5"],
```

```
         label="Moving Average (5)")
plt.plot(data.index, data["rolling_mean_10"],
         label="Moving Average (10)")
plt.xlabel("Timestamp")
plt.ylabel("Temperature")
plt.title("Original Series and Moving Averages")
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



[16]:
```
# 3. Plot rolling standard deviations
plt.figure(figsize=(12, 5))
plt.plot(data.index, data["rolling_std_5"],
         label="Rolling Std (5)")
plt.plot(data.index, data["rolling_std_10"],
         label="Rolling Std (10)")
plt.xlabel("Timestamp")
plt.ylabel("Rolling Standard Deviation")
plt.title("Rolling Standard Deviation")
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



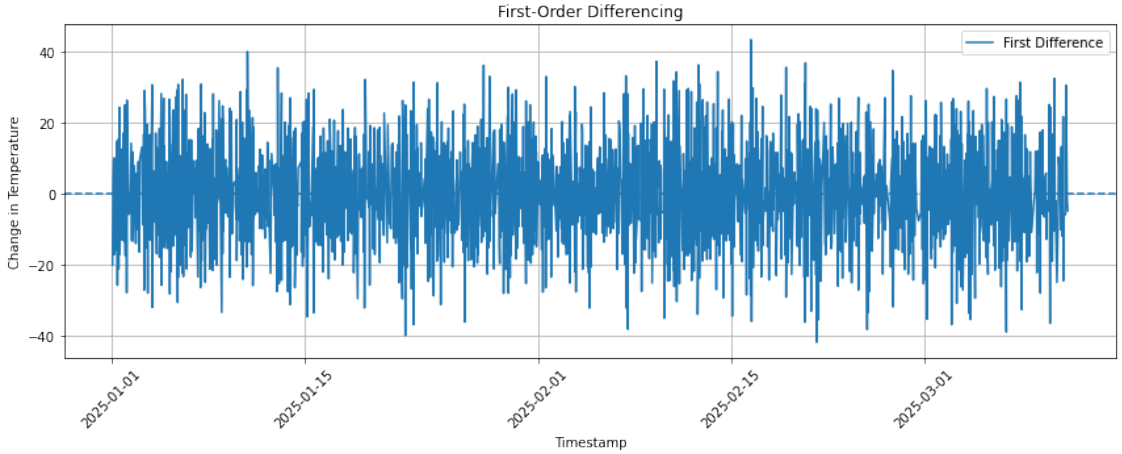
[17]:
```python
# 4. First-order differencing
data["first_difference"] = series.diff()

# 5. Percentage rate of change
data["percentage_change"] = series.pct_change(
    fill_method=None
) * 100

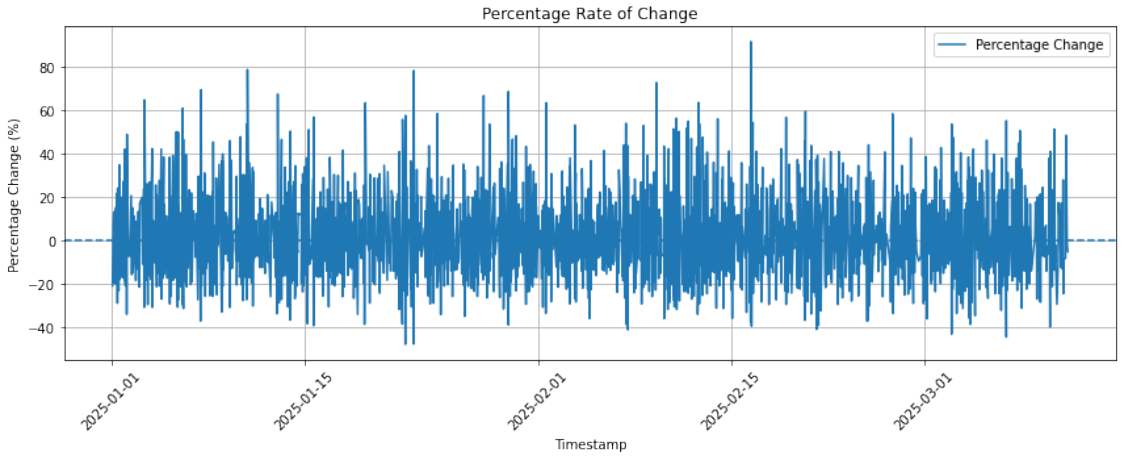
print("\nDifferencing and Percentage Change:")
print(data[[PARAMETER, "first_difference",
            "percentage_change"]].head(15))
```

```
Differencing and Percentage Change:
                     temperature  first_difference  percentage_change
timestamp
2025-01-01 00:08:00        96.04               NaN                NaN
2025-01-01 01:22:00        75.98            -20.06         -20.887130
2025-01-01 02:04:00        71.47             -4.51          -5.935773
2025-01-01 02:38:00        77.00              5.53           7.737512
2025-01-01 03:39:00        87.08             10.08          13.090909
2025-01-01 04:29:00        70.00            -17.08         -19.614148
2025-01-01 06:14:00        80.01             10.01          14.300000
2025-01-01 06:29:00        71.23             -8.78         -10.973628
2025-01-01 07:00:00        79.15              7.92          11.118911
2025-01-01 07:06:00        63.15            -16.00         -20.214782
2025-01-01 07:43:00        72.93              9.78          15.486936
2025-01-01 08:27:00        67.14             -5.79          -7.939120
2025-01-01 09:13:00        81.79             14.65          21.820077
2025-01-01 09:47:00        88.92              7.13           8.717447
2025-01-01 09:53:00        63.19            -25.73         -28.936122
```

[18]:
```
# 6. Plot first differences
plt.figure(figsize=(12, 5))
plt.plot(data.index, data["first_difference"],
         label="First Difference")
plt.axhline(y=0, linestyle="--")
plt.xlabel("Timestamp")
plt.ylabel("Change in Temperature")
plt.title("First-Order Differencing")
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



[20]:
```
# 7. Plot percentage change
plt.figure(figsize=(12, 5))
plt.plot(data.index, data["percentage_change"],
         label="Percentage Change")
plt.axhline(y=0, linestyle="--")
plt.xlabel("Timestamp")
plt.ylabel("Percentage Change (%)")
plt.title("Percentage Rate of Change")
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



5 Part C — Lag Analysis and Forecasting

[21]:
```
data["lag_1"] = series.shift(1)
data["lag_2"] = series.shift(2)
data["lag_3"] = series.shift(3)

print("Lag Variables:")
print(data[[PARAMETER, "lag_1", "lag_2", "lag_3"]].head(10))
```

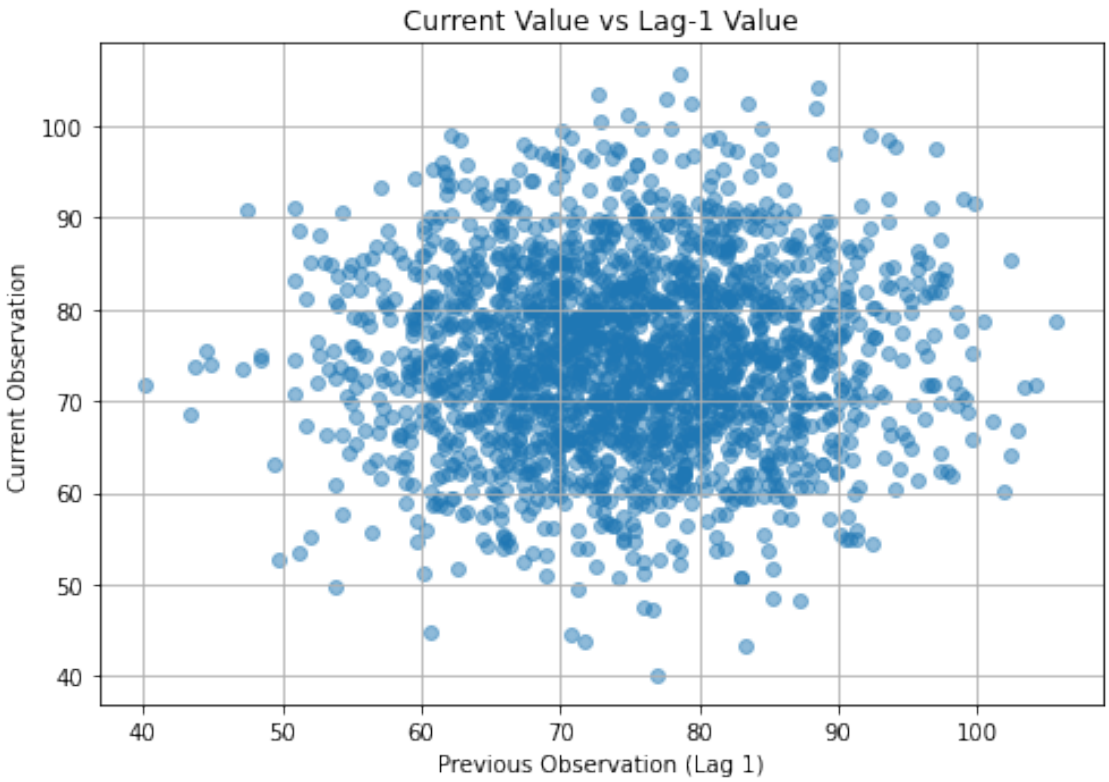
```
Lag Variables:
                     temperature  lag_1  lag_2  lag_3
timestamp
2025-01-01 00:08:00        96.04    NaN    NaN    NaN
2025-01-01 01:22:00        75.98  96.04    NaN    NaN
2025-01-01 02:04:00        71.47  75.98  96.04    NaN
2025-01-01 02:38:00        77.00  71.47  75.98  96.04
2025-01-01 03:39:00        87.08  77.00  71.47  75.98
2025-01-01 04:29:00        70.00  87.08  77.00  71.47
2025-01-01 06:14:00        80.01  70.00  87.08  77.00
2025-01-01 06:29:00        71.23  80.01  70.00  87.08
2025-01-01 07:00:00        79.15  71.23  80.01  70.00
2025-01-01 07:06:00        63.15  79.15  71.23  80.01
```

[23]:
```
# 3. Calculate lag correlations
print("\nLag Correlations:")
for lag in [1, 2, 3]:
    print(f"Lag {lag} correlation:", series.autocorr(lag=lag))
```

```
Lag Correlations:
Lag 1 correlation: 0.02272004792869126
```

```
Lag 2 correlation: -0.016079440225583913
Lag 3 correlation: 0.015164501145877246
```

[24]:
```python
# 4. Plot original series against lag-1 values
plt.figure(figsize=(7, 5))
plt.scatter(data["lag_1"], series, alpha=0.5)
plt.xlabel("Previous Observation (Lag 1)")
plt.ylabel("Current Observation")
plt.title("Current Value vs Lag-1 Value")
plt.grid(True)
plt.tight_layout()
plt.show()
```



[25]:
```python
# 5. Chronological train-test split
n = len(data)
split_index = int(n * 0.80)

if n < 10:
    raise ValueError("Not enough observations for forecasting.")

train = data.iloc[:split_index].copy()
test = data.iloc[split_index:].copy()
```

```
print("\nTraining observations:", len(train))
print("Testing observations:", len(test))
print("Test period:", test.index.min(), "to", test.index.max())
```

```
Training observations: 1673
Testing observations: 419
Test period: 2025-02-24 18:42:00 to 2025-03-11 10:36:00
```

[27]:
```
# 6. Generate one-step-ahead forecasts
# Naive forecast: previous observed value
data["naive_forecast"] = series.shift(1)

# Moving-average forecast: mean of the previous 5 observations
data["moving_average_forecast"] = (
    series.shift(1).rolling(window=5).mean()
)

# Select forecasts only for the chronological test period
results = data.iloc[split_index:][
    [PARAMETER, "naive_forecast", "moving_average_forecast"]
].copy()

# Evaluate both methods on the same timestamps
results = results.dropna()

if results.empty:
    raise ValueError("Not enough valid test forecasts to evaluate.")
```

[28]:
```
# 7. Calculate MAE and RMSE
actual = results[PARAMETER]

mae_naive = np.mean(
    np.abs(actual - results["naive_forecast"])
)
rmse_naive = np.sqrt(
    np.mean((actual - results["naive_forecast"]) ** 2)
)

mae_ma = np.mean(
    np.abs(actual - results["moving_average_forecast"])
)
rmse_ma = np.sqrt(
    np.mean((actual - results["moving_average_forecast"]) ** 2)
)
```

```
metrics = pd.DataFrame({
    "Forecast Method": ["Naive", "Moving Average (5)"],
    "MAE": [mae_naive, mae_ma],
    "RMSE": [rmse_naive, rmse_ma]
})

print("\nForecast Evaluation:")
print(metrics.to_string(index=False))
```

```
Forecast Evaluation:
   Forecast Method       MAE      RMSE
             Naive 11.339260 14.163605
Moving Average (5)  9.240912 11.268779
```

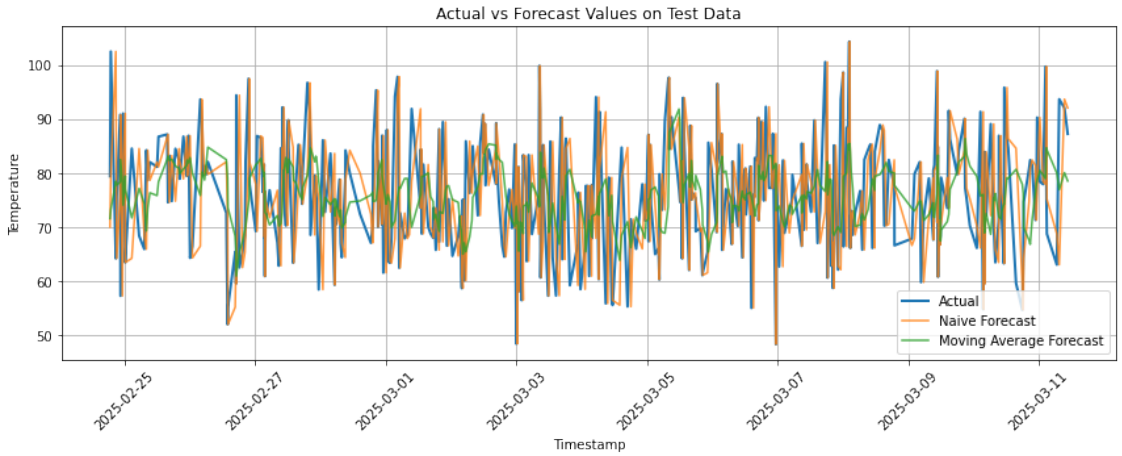
[29]:
```
# 8. Identify the better-performing method
if mae_naive < mae_ma:
    print("\nLower MAE: Naive forecast")
elif mae_ma < mae_naive:
    print("\nLower MAE: Moving Average forecast")
else:
    print("\nBoth methods have equal MAE")

if rmse_naive < rmse_ma:
    print("Lower RMSE: Naive forecast")
elif rmse_ma < rmse_naive:
    print("Lower RMSE: Moving Average forecast")
else:
    print("Both methods have equal RMSE")
```

```
Lower MAE: Moving Average forecast
Lower RMSE: Moving Average forecast
```

[30]:
```
# 9. Plot actual versus forecast values
plt.figure(figsize=(12, 5))
plt.plot(results.index, actual, label="Actual", linewidth=2)
plt.plot(results.index, results["naive_forecast"],
         label="Naive Forecast", alpha=0.8)
plt.plot(results.index, results["moving_average_forecast"],
         label="Moving Average Forecast", alpha=0.8)
plt.xlabel("Timestamp")
plt.ylabel("Temperature")
plt.title("Actual vs Forecast Values on Test Data")
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
```

```
plt.tight_layout()
plt.show()
```



[31]:
```
# 10. Display forecast results
print("\nSample Forecast Results:")
print(results.head(15))
```

```
Sample Forecast Results:
                     temperature  naive_forecast  moving_average_forecast
timestamp
2025-02-24 18:42:00        79.40           70.02                   71.588
2025-02-24 18:59:00       102.45           79.40                   73.174
2025-02-24 20:50:00        64.21          102.45                   78.472
2025-02-24 21:09:00        75.59           64.21                   77.784
2025-02-24 22:26:00        90.76           75.59                   78.334
2025-02-24 22:33:00        57.29           90.76                   82.482
2025-02-24 23:16:00        82.54           57.29                   78.060
2025-02-24 23:30:00        91.00           82.54                   74.078
2025-02-25 00:18:00        63.47           91.00                   79.436
2025-02-25 00:20:00        64.24           63.47                   77.012
2025-02-25 02:42:00        84.52           64.24                   71.708
2025-02-25 05:24:00        68.37           84.52                   77.154
2025-02-25 07:18:00        65.93           68.37                   74.320
2025-02-25 08:04:00        84.21           65.93                   69.306
2025-02-25 08:33:00        78.66           84.21                   73.454
```